UTM

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3133-01 High Performance Data Processing

Project 1: Optimizing High Performance Data Processing for Large Scale Web Crawlers

LECTURER: ASSOC. PROF. MOHD SHAHIZAN BIN OTHMAN

NO.	NAME	MATRIC NUMBER
1.	IMAN ABADI BINN MOHD NIZWAN	A23CS0084
2.	MOHAMED ALIF FATHI BIN ABDUL LATIF	A23CS0112
3.	DHESHIEGHAN A/L SARAVANA MOORTHY	A23CS0072

Table of Contents

1.0 Introduction

1.1 Background of the Project

Web scraping is a useful technique for collecting large amounts of data from websites automatically. In this project, web scraping is used to collect book product data from BookXcess, a Malaysian online bookstore with a large catalogue of books across different genres.

The project uses Beautiful Soup to parse the BookXcess XML sitemap and collect product URLs. Each product's structured data is then retrieved through its public *.json* endpoint. To improve scraping speed and reliability, the crawler uses *curl_cffi*, *ThreadPoolExecutor* with 40 workers, retry handling, and periodic CSV saving.

1.2 Objectives

The main goals of this project are:

- To develop a web scraper capable of collecting at least 100,000 product records from BookXcess.
- To clean and preprocess the raw dataset using pandas.
- To optimize data processing performance using Pandas, PyArrow and Polars and make an evaluation using bar charts with matplotlib.

1.3 Target Website and Data

The target website for this project is:

BookXcess
https://www.bookxcess.com/

BookXcess was selected because it provides a large number of product listings with detailed product information. The crawler collects product URLs by traversing the BookXcess XML sitemap index and product sub-sitemaps. After that, each product's data is extracted through its public *.json* endpoint.

The specific data fields extracted are:

Data Field	Description
product_id	Unique product identifier
title	Book title
product_type	Book author
vendor	Book publisher
sku	Product ISBN
current_price	Current selling price
compare_at_price	Original price before discount
currency	Product currency
grams	Product weight
tags	Product tags or category labels
body_html	Product description in HTML format
image_url	Book cover image URL
created_at	Product creation date on website

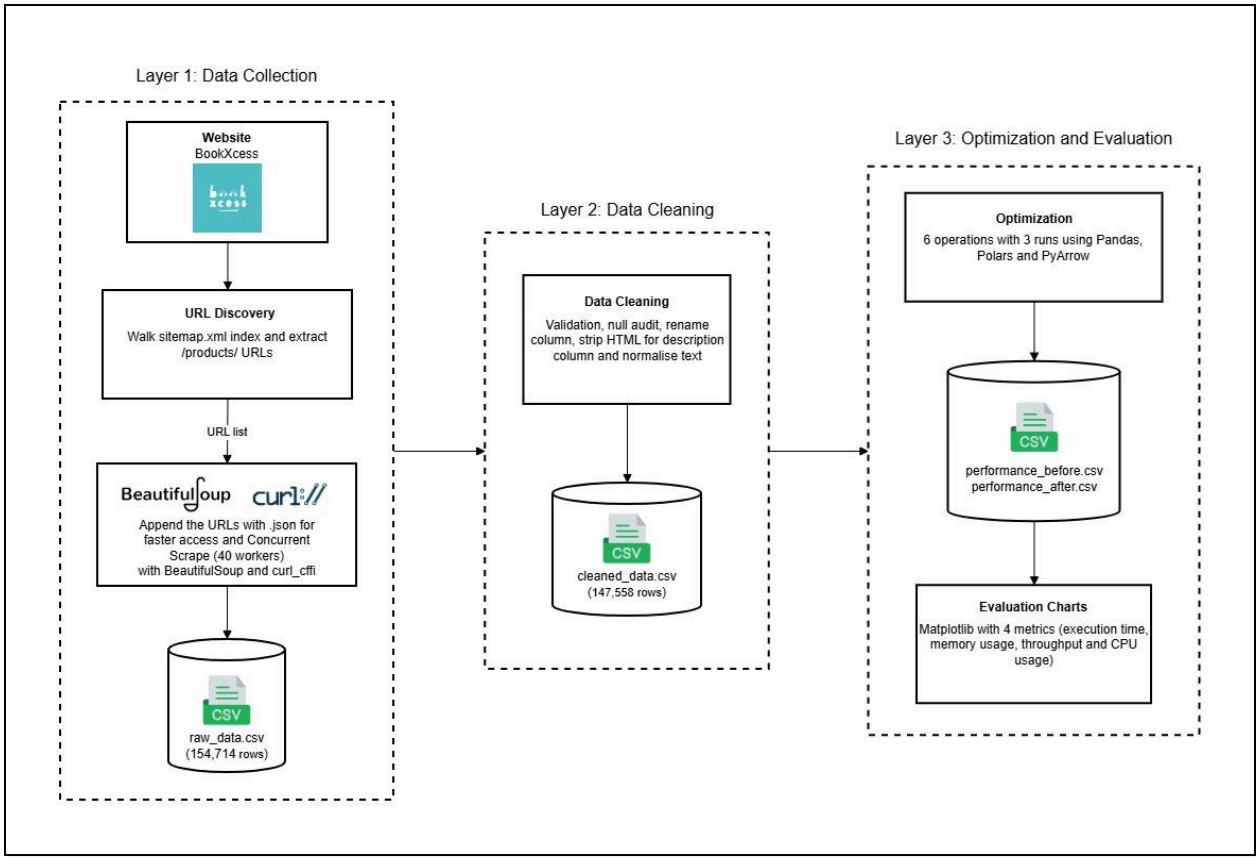
The dataset files used are:

- raw_data.csv: Raw data collected directly from the crawler (around 154,714 rows).
- cleaned_data.csv: Cleaned and structured dataset after preprocessing (around 147,558 rows).

2.0 System Design and Architecture

2.1 Pipeline Overview

The system is organized in three layers of process which are data collection, data cleaning and optimization and evaluation. There are four notebook files used in this project to execute data collection, perform data cleaning, run optimization benchmarks and generate evaluation visualizations respectively.



Layer 1: Data Collection

In this layer, notebook file *main_crawler.ipynb* scrapes data from website BookXcess by finding the URLs of the product using sitemaps. After finding all product URLs, the notebook appends the URLs with *.json* to access their product page information without loading longer when crawling. Web crawling will go to individual JSON links to parse all the book information from the website and stored in *raw_data.csv*.

Layer 2: Data Cleaning

In this layer, notebook file *clean_data.ipynb* ingests the raw data to perform validation, a null audit, column renaming, HTML stripping and text normalization. This step is important to refine the dataset into clean rows for the next operation which is optimization and evaluation. The cleaned data is then saved as cleaned_data.csv.

Layer 3: Optimization and Evaluation

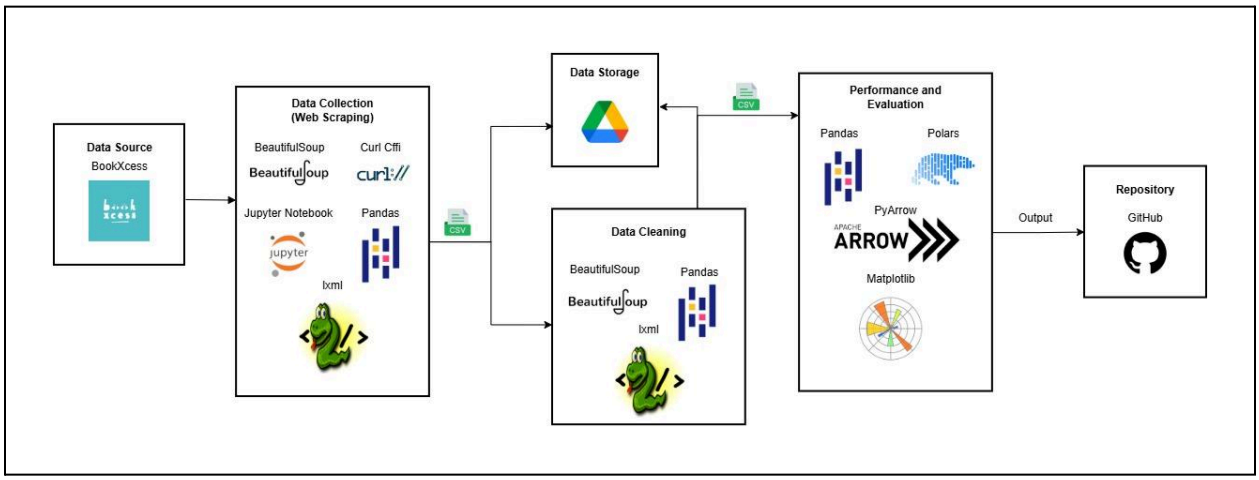
In this layer, the notebook files *optimize_pipeline.ipynb* execute six data operations with three separate runs (average) to benchmark Pandas, Polars and PyArrow. The results are then saved into performance CSV files (*performance_before.csv* and *performance_after.csv*). These CSV files are used to generate comparative charts using matplotlib with four metrics which are execution time (s), memory usage (mb), throughput (rows/s) and CPU usage in *evaluation_charts.ipynb*.

2.2 Tools and Frameworks

The technologies used are based on components below to achieve the pipeline goals.

Component	Tools and Description
HTTP client	Curl_cffi used to mimic a real browser's security handshake. This allows the crawler to seamlessly bypass anti-bot protections on BookXcess.
Concurrency	A ThreadPoolExecutor with 40 workers was implemented to optimize the I/O-bound crawling process. This allows multiple network requests to execute concurrently and significantly reduces scraping runtime.
Parsing	BeautifulSoup paired with the lxml parser used to extract clean text and structure data from raw HTML and XML sitemaps.
Cleaning	Pandas alongside html and unicodedata used to organize the scraped data, decode web text symbols and standardize formatting.
Optimization	Polars and PyArrow were used to benchmark advanced data processing speeds against Pandas.
Measurement	System resource usage tracked using psutil and native memory API to measure CPU load and memory consumption, while time.perf_counter measured execution speeds accurately.
Visualization	Matplotlib used to transform raw benchmarking data into visual comparison charts for each performance metric.

The diagram below shows the connection of the tools used through this project.



2.3 Team Roles

Work was distributed with three roles with every member contributing to coding, evaluation and documentation.

Role	Responsibilities	Member
Group Leader & Data Engineer	Coordinated the team, created the crawler and managed data ingestion and storage.	Mohamed Alif Fathi bin Abdul Latif
HPC & Performance Specialist	Designed and implemented the optimization benchmarks and produced the comparison charts.	Iman Abadi bin Mohd Nizwan
Analyst & Documentation Lead	Performed data cleaning and analysis and guided the report and slides.	Dheshieghan A/l Saravana Moorthy

3.0 Data Collection

3.1 Crawling Strategy

BookXcess publishes a sitemap index at /sitemap.xml that links to several product sub-sitemaps. The crawler works in two phases. First, a single discovery session walks the sitemap index, fetches each product sub-sitemap, extracts every /products/ URL and de-duplicates the list.

```
def extract_product_locs(xml_text: str) -> list:
    """Return only the page-level <loc> URLs that point to /products/ pages."""
    soup = BeautifulSoup(xml_text, "xml")
    urls = []
    for url_tag in soup.find_all("url"):
        loc = url_tag.find("loc", recursive=False)
        if loc and loc.text and "/products/" in loc.text:
            urls.append(loc.text.strip())
    return urls


def get_all_product_urls(session: cf_requests.Session) -> list:
    resp = session.get(SITEMAP_INDEX_URL, timeout=30)
    resp.raise_for_status()

    soup = BeautifulSoup(resp.text, "xml")
    sub_sitemaps = [
        tag.text.strip()
        for tag in soup.find_all("loc")
        if "products" in tag.text
    ]
    print(f"Found {len(sub_sitemaps)} product sub-sitemap(s).")

    product_urls = []
    for i, sitemap_url in enumerate(sub_sitemaps, 1):
        try:
            r = session.get(sitemap_url, timeout=30)
            r.raise_for_status()
            urls = extract_product_locs(r.text)
            product_urls.extend(urls)
            if i % 10 == 0 or i == len(sub_sitemaps):
                print(f"  [{i}/{len(sub_sitemaps)}] total URLs: {len(product_urls):,}")
        except Exception as exc:
            print(f"  Warning [{i}]: {sitemap_url.split('?')[0]} — {exc}")

        time.sleep(random.uniform(1.0, 1.8))

    # De-duplicate while preserving order
    seen, deduped = set(), []
    for u in product_urls:
        if u not in seen:
            seen.add(u)
            deduped.append(u)
    print(f"\nUnique product URLs: {len(deduped):,} (dropped {len(product_urls) - len(deduped):,} duplicates)")
    return deduped
```

When running the function *get_all_product_urls*, BeautifulSoap will parse all the URLs of product sub-sitemaps. The image below shows the example of the products' sub-sitemap structure.

```
<sitemap>
  <loc>https://www.bookxcess.com/sitemap_products_1.xml?from=6978034335950&to=6978117959886</loc>
</sitemap>
<sitemap>
  <loc>https://www.bookxcess.com/sitemap_products_2.xml?from=6978117992654&to=6978203877582</loc>
</sitemap>
<sitemap>
  <loc>https://www.bookxcess.com/sitemap_products_3.xml?from=6978203910350&to=6978289008846</loc>
</sitemap>
```

After finding all the sub-sitemap links, the scraper will go through the sub-sitemap links and parse the URL link to the individual books website page using *extract_product_locs* function. The image below shows the example of the products list in the sub-sitemap structure.

```
<url>
  <loc>https://www.bookxcess.com/products/one-summer</loc>
  <lastmod>2026-05-25T02:45:16+08:00</lastmod>
  <changefreq>daily</changefreq>
  <image:image>
    <image:loc>https://cdn.shopify.com/s/files/1/0601/0883/2974/products/9780446583152.jpg?v=1648781165</image:loc>
    <image:title>One Summer</image:title>
    <image:caption/>
  </image:image>
</url>
<url>
  <loc>https://www.bookxcess.com/products/hunting-season</loc>
  <lastmod>2026-05-25T02:45:16+08:00</lastmod>
  <changefreq>daily</changefreq>
  <image:image>
    <image:loc>https://cdn.shopify.com/s/files/1/0601/0883/2974/products/hunting-season-9781447265931-3462335037484.jpg?
    v=1648781165</image:loc>
    <image:title>Hunting Season</image:title>
    <image:caption>Hunting Season</image:caption>
  </image:image>
</url>
<url>
```

The second phase is when URLs are scraped concurrently.

```python
def fetch_product_data(product_url: str) -> dict:
    """Fetch one product, retrying transient errors. Returns 13 raw fields."""
    json_url = product_url.rstrip("/") + ".json"
    last_exc = None

    for attempt in range(MAX_RETRIES):
        try:
            resp = get_session().get(json_url, timeout=30)
            if resp.status_code in TRANSIENT_STATUSES:
                raise RuntimeError(f"HTTP {resp.status_code}")
            resp.raise_for_status()

            product = resp.json()["product"]
            variant = product["variants"][0] if product.get("variants") else {}
            image   = product.get("image") or {}

            return {
                "Product ID"       : product.get("id"),
                "Title"            : product.get("title"),
                "Product Type"     : product.get("product_type"),
                "Vendor"           : product.get("vendor"),
                "SKU"              : variant.get("sku"),
                "Current Price"    : variant.get("price"),
                "Compare at Price" : variant.get("compare_at_price"),
                "Currency"         : variant.get("price_currency"),
                "Grams"            : variant.get("grams"),
                "Tags"             : product.get("tags"),
                "Body HTML"        : product.get("body_html"),
                "Image URL"        : image.get("src"),
                "Created At"       : product.get("created_at"),
            }
        except Exception as exc:
            last_exc = exc
            if attempt < MAX_RETRIES - 1:
                backoff = random.uniform(*RETRY_BACKOFF) * (2 ** attempt)
                time.sleep(backoff)

    raise last_exc
```

Each product URL is converted to its Shopify JSON form by appending .json with *fetch_product_data* function. This can be done because the website (BookXcess) is built using Shopify which is an e-commerce platform. The crawler reads the variant, image and product and then records the thirteen fields. Working with JSON rather than rendered HTML makes extraction faster and far more robust. The image below shows the JSON file structure of a book page.

```
{
  "product": {
    "id": 6978034335950,
    "title": "One Summer",
    "body_html": "\u003Cp\u003EIt's almost Christmas, but there is no joy in the house of terminally ill Jack and his family. With only a short time left to live, he spends his last days preparing to say goodbye to his devoted wife, Lizzie, and their three children. Then, unthinkably, tragedy strikes again: Lizzie is killed in a car accident. With no one able to care for them, the children are separated from each other and sent to live with family members around the country. Just when all seems lost, Jack begins to recover in a miraculous turn of events. He rises from what should have been his deathbed, determined to bring his fractured family back together. Struggling to rebuild their lives after Lizzie's death, he reunites everyone at Lizzie's childhood home on the oceanfront in South Carolina. And there, over one unforgettable summer, Jack will begin to learn to love again, and he and his children will learn how to become a family once more.\u003C/p\u003E",
    "vendor": "Grand Central Publishing",
    "product_type": "David Baldacci",
    "created_at": "2022-04-01T10:46:04+08:00",
    "handle": "one-summer",
    "updated_at": "2026-05-25T03:03:51+08:00",
    "published_at": "2020-10-02T12:07:44+08:00",
    "template_suffix": "",
    "published_scope": "web",
    "tags": "[2022], BXOS, David Baldacci, F GEN, Fiction, General Fiction, Paperback, RM 10 - RM 19.99, Teenage / Young Adult",
    "variants": [
      {
        "id": 41077803909326,
        "product_id": 6978034335950,
        "title": "Default Title",
        "price": "17.90",
        "sku": "9780446583152",
        "position": 1,
        "compare_at_price": "80.96",
        "fulfillment_service": "manual",
        "inventory_management": "shopify",
        "option1": "Default Title",
        "option2": null,
        "option3": null,
        "created_at": "2022-04-01T10:46:05+08:00",
        "updated_at": "2026-05-25T03:03:51+08:00",
        "taxable": true,
        "barcode": "9780446583152",
        "grams": 320,
        "image_id": null,
        "weight": 320,
        "weight_unit": "g",
        "requires_shipping": true,
        "quantity_rule": {
          "min": 1,
          "max": null,
          "increment": 1
        },
        "price_currency": "MYR",
        "compare_at_price_currency": "MYR",
        "quantity_price_breaks": []
      }
    ],
    "options": [
```

3.2 Crawling Optimization

```
MAX_WORKERS       = 40
```

```
with ThreadPoolExecutor(max_workers=MAX_WORKERS) as pool:
    futures = {pool.submit(fetch_with_jitter, u): u for u in product_urls}
```

Data collection is driven by a 40-worker *ThreadPoolExecutor*. To ensure thread safety and prevent connection conflicts, each worker maintains an independent curl_cffi session via thread-local storage. Because the web crawling process is heavily I/O-bound, a multithreaded architecture is highly effective. This allows the pipeline to maximize throughput because of the overlapping idle network latency across multiple concurrent requests.

3.3 Reliability and Ethical Crawling

There are several measures to keep the crawl both resilient and respectful of the target server.

1. Per-request jitter

```
REQUEST_JITTER    = (0.05, 0.2)
```

```
def fetch_with_jitter(product_url: str) -> dict:
    time.sleep(random.uniform(*REQUEST_JITTER))
    return fetch_product_data(product_url)
```

   Per-request jitter (random delay duration between 50 milliseconds and 200 milliseconds) smooths out the request pattern instead of bursting the website.

2. HTTP errors checking

```
TRANSIENT_STATUSES = (429, 500, 502, 503, 504)
```

```
MAX_RETRIES       = 3
RETRY_BACKOFF     = (1.5, 3.5)
```

```
try:
    resp = get_session().get(json_url, timeout=30)
    if resp.status_code in TRANSIENT_STATUSES:
        raise RuntimeError(f"HTTP {resp.status_code}")
    resp.raise_for_status()
```

```
except Exception as exc:
    last_exc = exc
    if attempt < MAX_RETRIES - 1:
        backoff = random.uniform(*RETRY_BACKOFF) * (2 ** attempt)
        time.sleep(backoff)
```

Temporary HTTP errors (429, 500, 502, 503, 504) and connection failures are retried up to three times with randomised exponential backoff (between 1.5 and 3.5 seconds).

3. Backup CSV while scraping

```
def flush_partial():
    with flush_lock:
        pd.DataFrame(results).to_csv(PARTIAL_FILE, index=False, encoding="utf-8-sig")
```

```
if i % FLUSH_EVERY == 0:
    flush_partial()
    print(f"  ... flushed partial CSV ({len(results):,} rows)")
```

Progress is flushed to a partial CSV every 5,000 successful products using flush_patial function. This makes a crash never lose more than a few thousand rows of work.

4. Rescrape failed URLs

```
if failed_urls:
    retry_targets = [u for u, _ in failed_urls]
    recovered     = []
    still_failed  = []

    with ThreadPoolExecutor(max_workers=max(MAX_WORKERS // 2, 5)) as pool:
        futures = {pool.submit(fetch_with_jitter, u): u for u in retry_targets}
        for fut in as_completed(futures):
            url = futures[fut]
            try:
                recovered.append(fut.result())
            except Exception as exc:
                still_failed.append((url, str(exc)))

    print(f"Recovered: {len(recovered):,}   Still failed: {len(still_failed):,}")
    if recovered:
        df_full = pd.concat([df, pd.DataFrame(recovered)], ignore_index=True)
        df_full.to_csv(OUTPUT_FILE, index=False, encoding="utf-8-sig")
        print(f"Updated '{OUTPUT_FILE}' — now {len(df_full):,} rows.")
    if still_failed:
        pd.DataFrame(still_failed, columns=["URL", "Error"]).to_csv(
            FAILED_FILE, index=False, encoding="utf-8-sig"
        )
else:
    print("No failed URLs to retry.")
```

URLs that fail are written to a separate file and can be rescraped with a gentler format by having a half-sized worker pool. The failed rows then append to the successful scraped dataframe.

3.4 Result

```
df = pd.DataFrame(results)
df.to_csv(OUTPUT_FILE, index=False, encoding="utf-8-sig")

if failed_urls:
    pd.DataFrame(failed_urls, columns=["URL", "Error"]).to_csv(
        FAILED_FILE, index=False, encoding="utf-8-sig"
    )
    print(f"Saved {len(failed_urls):,} failed URLs to '{FAILED_FILE}'")

if os.path.exists(PARTIAL_FILE):
    try:
        os.remove(PARTIAL_FILE)
    except Exception:
        pass

print(f"\nSaved {len(df):,} rows to '{OUTPUT_FILE}'\n")
print("Field completeness:")
print(df.notna().sum())
df.head(5)
```

The crawl produced a raw dataset of product records written to *raw_data.csv* with total rows is 154,714. Field completeness was inspected immediately after the run before handing off to the cleaning stage.

4.0 Data Processing and Transformation

This section explains the processing and transforming steps done to clean and standardize the columns of the dataset.

4.1 Drop Missing Values

```
INPUT_FILE  = "raw_data.csv"
```

```
df = pd.read_csv(INPUT_FILE)

print("Shape:", df.shape)
print()
print("Data types:")
print(df.dtypes)
print()
print("Missing values per column:")
print(df.isna().sum())
print()
df.head(3)
```

Before cleaning, the raw dataset is loaded using pandas and checked for missing values. Rows with missing values are removed to make sure the dataset is complete before further processing. This ensures that incomplete product records are removed from the dataset.

4.2 Column Name Standardization

```
df = df.rename(columns={
    "Product ID":       "product_id",
    "Title":            "title",
    "Product Type":     "author",
    "Vendor":           "publisher",
    "SKU":              "sku",
    "Current Price":    "current_price",
    "Compare at Price": "compare_price",
    "Currency":         "currency",
    "Grams":            "grams",
    "Tags":             "tags",
    "Body HTML":        "body_html",
    "Image URL":        "image_url",
    "Created At":       "created_at",
})
```

The raw dataset contains column names such as *Product ID*, *Title*, *Product Type*, *Vendor*, *Current Price*, *Compare at Price*, *Body HTML* and *Created At*. These columns are renamed into cleaner and more consistent names using lowercase letters and underscores.

```
assert df["currency"].dropna().nunique() == 1, "Expected only MYR"
df = df.drop(columns=["currency"])
```

The Currency column is removed because all rows use the same currency, which is *MYR*.

4.3 Date Formatting

```
df["created_at"] = pd.to_datetime(df["created_at"], errors="coerce")
```

The *created_at column* is converted into datetime format so that the product creation date can be processed and analysed properly. This makes the date field more consistent and suitable for future date-based analysis.

4.4 HTML Cleaning

```
def html_to_text(value):
    """Strip HTML tags, return plain text. Returns None for missing values."""
    if pd.isna(value):
        return None
    soup = BeautifulSoup(value, "lxml")
    text = soup.get_text(separator=" ")
    text = re.sub(r"\s+", " ", text)
    return text.strip() or None


df["description"] = df["body_html"].apply(html_to_text)
df = df.drop(columns=["body_html"])
```

The product description is originally stored in the Body HTML column. Since it contains HTML tags, Beautiful Soup is used to convert it into plain readable text. This removes HTML elements and creates a cleaner description column.

4.5 Title Cleaning

```
_BARGAIN_RE = re.compile(r"^\[bargain\s+corner\]\s*", re.IGNORECASE)

before_count = df["title"].str.contains(_BARGAIN_RE, na=False).sum()
print(f"Titles with '[Bargain Corner]' prefix before cleaning: {before_count}")

df["title"] = df["title"].apply(
    lambda v: _BARGAIN_RE.sub("", v).strip() if isinstance(v, str) else v
)
```

Some product titles contain the prefix *[Bargain Corner]*. This prefix is removed to keep the book titles clean and consistent.

4.6 Text Normalization

```python
def clean_text(value):
    """Decode entities, normalize unicode, strip recurring ? and tidy whitespace."""
    if pd.isna(value):
        return value
    text = str(value)
    # Normalize unicode (full-width → half-width, ligatures, etc.)
    text = unicodedata.normalize("NFKC", text)
    # Decode &amp;, &quot;, &#10;, etc.
    text = html.unescape(text)
    # Replace newline / carriage-return / tab / non-breaking-space with a regular space
    text = text.replace("\n", " ").replace("\r", " ").replace("\t", " ")
    text = text.replace("\u00a0", " ")
    # Remove runs of 2+ consecutive question marks (data artefact)
    text = re.sub(r"\?{2,}", "", text)
    # Collapse whitespace and strip ends
    text = re.sub(r"\s+", " ", text).strip()
    return text or None
```

Text columns are cleaned by decoding HTML entities, normalizing Unicode characters, removing unwanted symbols, removing newlines or tabs and standardizing spacing.

```python
TEXT_COLUMNS = ["title", "author", "publisher", "sku", "tags", "image_url", "description"]

for col in TEXT_COLUMNS:
    df[col] = df[col].apply(clean_text)
```

This cleaning function is applied to these text-based columns.

4.7 Author and Publisher Validation

The author and publisher fields are checked to make sure they contain valid names. Dangling punctuation is removed, and rows with invalid author or publisher values are dropped.

```python
_DANGLING_RE = re.compile(r"^[\s!,;:.\-&]+|[\s!,;:.\-&]+$")


def strip_dangling_punctuation(value):
    """Remove leading/trailing stray punctuation characters."""
    if pd.isna(value):
        return value
    cleaned = _DANGLING_RE.sub("", value).strip()
    return cleaned if cleaned else None


def is_valid_name(value):
    """Return True if value contains at least one letter. NaN is treated as valid (missing ≠ bad)."""
    if pd.isna(value):
        return True
    return bool(re.search(r"[a-zA-Z]", value))


# Strip dangling punctuation first, then check validity.
df["author"]    = df["author"].apply(strip_dangling_punctuation)
df["publisher"] = df["publisher"].apply(strip_dangling_punctuation)

invalid_author    = ~df["author"].apply(is_valid_name)
invalid_publisher = ~df["publisher"].apply(is_valid_name)
invalid_either    = invalid_author | invalid_publisher

print("Rows with invalid author:   ", int(invalid_author.sum()))
print("Rows with invalid publisher:", int(invalid_publisher.sum()))
print("Rows dropped (either):      ", int(invalid_either.sum()))

if invalid_either.sum() > 0:
    print("\nSamples of dropped rows:")
    print(df.loc[invalid_either, ["title", "author", "publisher"]].head(10).to_string())

before = len(df)
df = df[~invalid_either].reset_index(drop=True)
print(f"\nDropped {before - len(df)} rows. Remaining: {len(df)}")
```

This ensures that invalid text values are removed from important fields.

4.8 Price Logic Cleaning and Discount Calculation

Rows with illogical prices are removed. This includes rows where the compare price is lower than the current price, and rows where the compare price is zero.

```
negative_discount = df["compare_price"] < df["current_price"]
zero_compare      = df["compare_price"] == 0
bad_rows          = negative_discount | zero_compare

print("Rows where compare_price < current_price (negative discount):", int(negative_discount.sum()))
print("Rows where compare_price == 0 (undefined discount):          ", int(zero_compare.sum()))
print("Total illogical rows to drop:                                ", int(bad_rows.sum()))

if bad_rows.sum() > 0:
    print("\nSamples of dropped rows:")
    print(df.loc[bad_rows, ["title", "current_price", "compare_price"]].head(10).to_string())

before = len(df)
df = df[~bad_rows].reset_index(drop=True)
print(f"\nDropped {before - len(df)} rows. Remaining: {len(df)}")

# Calculate discount (safe now — compare_price is either >= current_price or NaN)
df["discount_pct"] = (
    (df["compare_price"] - df["current_price"]) / df["compare_price"] * 100
).round(1)
```

4.9 Final Quality Checks

Hard checks are applied to make sure the final dataset is clean and logical.

```
# --- Hard assertions ---
assert (df["current_price"] >= 0).all(), \
    "Found negative current_price"
assert (df["compare_price"].dropna() > 0).all(), \
    "Found zero or negative compare_price"
assert (df["discount_pct"].dropna() >= 0).all(), \
    "Found negative discount_pct"
assert (df["discount_pct"].dropna() <= 100).all(), \
    "Found discount_pct > 100 (impossible)"
assert df["title"].str.contains(r"^\[Bargain", case=False, na=False).sum() == 0, \
    "Some titles still start with [Bargain Corner]"
assert df["author"].dropna().apply(is_valid_name).all(), \
    "Found author values with no letters"
assert df["publisher"].dropna().apply(is_valid_name).all(), \
    "Found publisher values with no letters"
assert (df["created_at"].notna()).all(), \
    "Some created_at values are missing"

print("All hard checks passed.")
print()
print("Final shape:", df.shape)
print("Final columns:", df.columns.tolist())
print()
df.dropna(inplace=True) # Final safety check: drop any remaining rows with missing values.
print("Missing values per column:")
print(df.isna().sum())
print()
df.head(3)
```

5.0 Optimisation Techniques

Three different data processing libraries were used to optimize the pipeline architecture which were Pandas, Polars and PyArrow to compare and evaluate the performance of each library with a total of six different operations. These operations were benchmarked in terms of execution time(s), memory used (MB), throughput (records/s) and CPU utilization (%).

Operations used for benchmarking:

- Filter by price and discount
- Group by publisher with aggregation
- String extraction from tags column
- Derive savings column
- Sort by discount percentage and current price
- Top authors by average discounts

5.1 Code Implementation

This section describes the core implementation and techniques used for benchmarking each library using the operations mentioned.

1. Memory flushing and in-memory measurement functions

```python
def flush_memory():
    gc.collect()
    time.sleep(1.0)


def result_size_mb(obj):
    """Return the in-memory footprint of a result object in MB using each library's native API."""
    try:
        if isinstance(obj, (pd.DataFrame, pd.Series)):
            return obj.memory_usage(deep=True).sum() / (1024 ** 2)
        elif isinstance(obj, (pl.DataFrame, pl.Series)):
            return obj.estimated_size() / (1024 ** 2)
        elif isinstance(obj, (pa.Table, pa.ChunkedArray, pa.Array)):
            return obj.nbytes / (1024 ** 2)
        else:
            return sys.getsizeof(obj) / (1024 ** 2)
    except Exception:
        return 0.0
```

The **flush_memory()** function is for garbage collection and sleeps for one second to ensure a clean state before proceeding with the next benchmarking session. The

result_size_mb() function measures the in-memory size of a result using each library's native API for more accurate measurements.

2. Benchmarking function

```python
def benchmark(operation_name, library_name, func, total_rows, runs=3):
    """
    Run func `runs` times with memory flushing between runs.
    Returns a list of dicts: one row per run (1..runs) plus one
    trailing 'average' row. Memory tracked via result-object size (library native API).
    CPU is captured both before (initial) and after (final) the operation.
    """
    records = []
    exec_times, cpu_initials, cpu_finals, mems = [], [], [], []

    for run_idx in range(1, runs + 1):
        flush_memory()

        # Clean pre-operation CPU reading (after the 1-second sleep in flush_memory)
        cpu_initial = psutil.cpu_percent(interval=None)

        start  = time.perf_counter()
        result = func()
        if hasattr(result, "compute"):
            result = result.compute()
        end = time.perf_counter()

        cpu_final = psutil.cpu_percent(interval=1)  # post-operation reading
        t   = end - start
        mem = result_size_mb(result)
        tput = total_rows / t if t > 0 else 0

        records.append({
            "operation": operation_name,
            "library": library_name,
            "run": run_idx,
            "execution_time_sec": round(t, 6),
            "cpu_initial_percent": round(cpu_initial, 2),
            "cpu_final_percent": round(cpu_final, 2),
            "memory_used_mb": round(mem, 3),
            "throughput_rows_per_sec": round(tput, 0),
        })
        exec_times.append(t)
        cpu_initials.append(cpu_initial)
        cpu_finals.append(cpu_final)
        mems.append(mem)

        del result
        flush_memory()

    avg_t = float(np.mean(exec_times))
    records.append({
        "operation": operation_name,
        "library": library_name,
        "run": "average",
        "execution_time_sec": round(avg_t, 6),
        "cpu_initial_percent": round(float(np.mean(cpu_initials)), 2),
        "cpu_final_percent": round(float(np.mean(cpu_finals)), 2),
        "memory_used_mb": round(float(np.mean(mems)), 3),
        "throughput_rows_per_sec": round(total_rows / avg_t if avg_t > 0 else 0, 0),
    })
    return records
```

The **benchmark()** is the core implementation measuring all the metrics based on the operations done. It utilizes the **flush_memory()** and **result_size_mb()** for each operation given. Each operation is run three times and is averaged to prevent false readings and inaccuracies.

3. Filter high-discount books operation

Pandas:

```
# 1. Filter high-discount books
results.extend(benchmark(
    "Filter High Discount Books", "Pandas",
    lambda: pdf[(pdf["discount_pct"] > 70) & (pdf["current_price"] < 20)],
    total_rows
))
```

Polars:

```
# 1. Filter high-discount books
results.extend(benchmark(
    "Filter High Discount Books", "Polars",
    lambda: pl_df.filter(
        (pl.col("discount_pct") > 70) & (pl.col("current_price") < 20)
    ),
    total_rows
))
```

PyArrow:

```
# 1. Filter high-discount books
results.extend(benchmark(
    "Filter High Discount Books", "PyArrow",
    lambda: pa_table.filter(
        pc.and_(
            pc.greater(pa_table.column("discount_pct"), 70),
            pc.less(pa_table.column("current_price"), 20)
        )
    ),
    total_rows
))
```

This is the code implementation for all three to filter high-discount books where it utilizes the columns *discount_pct* and *current_price* where *discount_pct* is more than 70 percent and *current_price* is less than RM 20.

4. Group by publisher aggregation

Pandas:

```
# 2. GroupBy publisher aggregation
results.extend(benchmark(
    "GroupBy Publisher Aggregation", "Pandas",
    lambda: pdf.groupby("publisher").agg({
        "current_price": "mean",
        "discount_pct": "mean",
        "title": "count"
    }),
    total_rows
))
```

Polars:

```
# 2. GroupBy publisher aggregation
results.extend(benchmark(
    "GroupBy Publisher Aggregation", "Polars",
    lambda: pl_df.group_by("publisher").agg([
        pl.col("current_price").mean(),
        pl.col("discount_pct").mean(),
        pl.col("title").count(),
    ]),
    total_rows
))
```

PyArrow:

```
# 2. GroupBy publisher aggregation
results.extend(benchmark(
    "GroupBy Publisher Aggregation", "PyArrow",
    lambda: pa_table.group_by("publisher").aggregate([
        ("current_price", "mean"),
        ("discount_pct", "mean"),
        ("title", "count"),
    ]),
    total_rows
))
```

This is the code implementation using aggregation to calculate mean *current_price*, mean *discount_ price* and count the number of book *title* then grouped by publisher.

5. String extraction price tier operation

Pandas:

```
# 3. String extract price tier
results.extend(benchmark(
    "String Extract Price Tier", "Pandas",
    lambda: pdf["tags"].astype(str).str.extract(r'(RM[\d\s\-\.]+)'),
    total_rows
))
```

Polars:

```
# 3. String extract price tier
results.extend(benchmark(
    "String Extract Price Tier", "Polars",
    lambda: pl_df.select(
        pl.col("tags").str.extract(r'(RM[\d\s\-\.]+)', group_index=1)
    ),
    total_rows
))
```

PyArrow:

```
# 3. String extract price tier (named capture group required by PyArrow)
results.extend(benchmark(
    "String Extract Price Tier", "PyArrow",
    lambda: pc.extract_regex(
        pc.cast(pa_table.column("tags"), pa.string()),
        pattern=r'(?P<price_tier>RM[\d\s\-\.]+)'
    ),
    total_rows
))
```

This is the code implementation that uses regex to extract “RM” prefixed in the *tags* column

6. Derived savings column calculation operation

Pandas:

```
# 4. Derived column calculation
results.extend(benchmark(
    "Calculate Savings", "Pandas",
    lambda: pdf.assign(
        savings_rm=pdf["compare_price"] - pdf["current_price"],
        savings_pct_check=(
            (pdf["compare_price"] - pdf["current_price"]) / pdf["compare_price"] * 100
        ).round(1)
    ),
    total_rows
))
```

Polars:

```
# 4. Derived column calculation
results.extend(benchmark(
    "Calculate Savings", "Polars",
    lambda: pl_df.with_columns([
        (pl.col("compare_price") - pl.col("current_price")).alias("savings_rm"),
        ((pl.col("compare_price") - pl.col("current_price"))
         / pl.col("compare_price") * 100).round(1).alias("savings_pct_check"),
    ]),
    total_rows
))
```

PyArrow:

```
# 4. Derived column calculation
def _pyarrow_savings():
    savings_rm = pc.subtract(pa_table.column("compare_price"), pa_table.column("current_price"))
    savings_pct = pc.round(
        pc.multiply(
            pc.divide(savings_rm, pa_table.column("compare_price")),
            100
        ),
        1
    )
    return (
        pa_table
        .append_column("savings_rm", savings_rm)
        .append_column("savings_pct_check", savings_pct)
    )

results.extend(benchmark("Calculate Savings", "PyArrow", _pyarrow_savings, total_rows))
```

This is the code implementation that calculates the *savings_rm* by subtracting *compare_price* and *current_price* then calculating *savings_rm*.

7. Sorting by discount and price operation

Pandas:

```
# 5. Sort by discount and price
results.extend(benchmark(
    "Sort by Discount and Price", "Pandas",
    lambda: pdf.sort_values(["discount_pct", "current_price"], ascending=[False, True]),
    total_rows
))
```

Polars:

```
# 5. Sort by discount and price
results.extend(benchmark(
    "Sort by Discount and Price", "Polars",
    lambda: pl_df.sort(
        by=["discount_pct", "current_price"], descending=[True, False]
    ),
    total_rows
))
```

PyArrow:

```
# 5. Sort by discount and price
results.extend(benchmark(
    "Sort by Discount and Price", "PyArrow",
    lambda: pa_table.take(
        pc.sort_indices(
            pa_table,
            sort_keys=[("discount_pct", "descending"), ("current_price", "ascending")]
        )
    ),
    total_rows
))
```

This implementation uses sorting operations to benchmark each library's capabilities. The sorted columns were *discount_pct* and *current_price* where they are sorted in descending order and ascending order respectively.

8. Identifying top 20 authors by average discounts operation

Pandas:

```
# 6. Top authors by average discount
results.extend(benchmark(
    "Top Authors by Average Discount", "Pandas",
    lambda: pdf.groupby("author")["discount_pct"].mean().sort_values(ascending=False).head(20),
    total_rows
))
```

Polars:

```
# 6. Top authors by average discount
results.extend(benchmark(
    "Top Authors by Average Discount", "Polars",
    lambda: pl_df.group_by("author").agg(
        pl.col("discount_pct").mean().alias("discount_pct_mean")
    ).sort("discount_pct_mean", descending=True).head(20),
    total_rows
))
```

PyArrow:

```
# 6. Top authors by average discount
def _pyarrow_top_authors():
    grp = pa_table.group_by("author").aggregate([("discount_pct", "mean")])
    idx = pc.sort_indices(grp, sort_keys=[("discount_pct_mean", "descending")])
    return grp.take(idx[:20])

results.extend(benchmark("Top Authors by Average Discount", "PyArrow", _pyarrow_top_authors, total_rows))
```

This implementation first is done by calculating mean discount_pct, sorting them in descending order, grouped by authors and then taking only the first 20 records.

9. Output CSV file

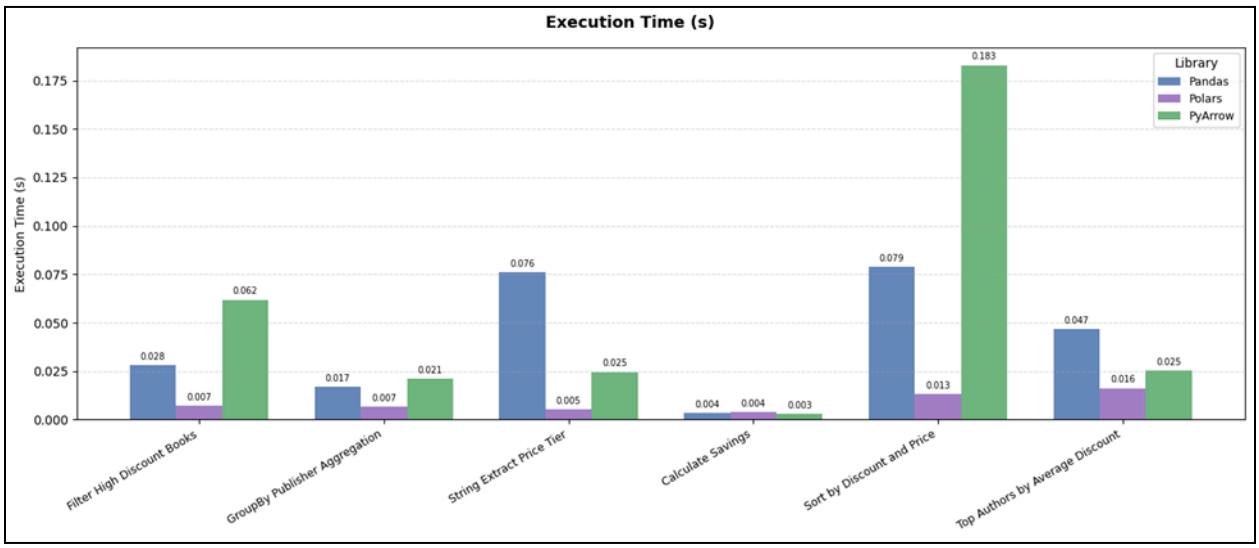
```
results_df = pd.DataFrame(results)

pandas_only = results_df[results_df["library"] == "Pandas"]
pandas_only.to_csv("performance_before.csv", index=False)

results_df.to_csv("performance_after.csv", index=False)

print("Saved performance_before.csv and performance_after.csv")
results_df
```

All of the processes are then saved in two CSV files where the performance_before.csv file contains only the readings from the Pandas implementation and performance_after.csv contains readings of all library operations.

10. Evaluation charts from benchmarking outputs

```
LIBRARIES = ["Pandas", "Polars", "PyArrow"]
COLORS    = ["#4C72B0", "#9467BD", "#55A868"]  # blue, purple, green
METRICS   = [
    ("execution_time_sec",      "Execution Time (s)"),
    ("memory_used_mb",          "Memory Used (MB)"),
    ("throughput_rows_per_sec", "Rows / Second"),
    ("cpu_final_percent",       "CPU Usage (%)"),
]

avg_df     = after_df[after_df["run"] == "average"].copy()
operations = avg_df["operation"].unique().tolist()

x     = np.arange(len(operations))  # 6 tick positions
width = 0.25                         # width of each individual bar

for metric, title in METRICS:
    fig, ax = plt.subplots(figsize=(14, 6))
    fig.suptitle(title, fontsize=13, fontweight="bold")

    for i, (lib, color) in enumerate(zip(LIBRARIES, COLORS)):
        vals = [
            avg_df.loc[
                (avg_df["operation"] == op) & (avg_df["library"] == lib), metric
            ].values[0]
            if ((avg_df["operation"] == op) & (avg_df["library"] == lib)).any() else 0
            for op in operations
        ]
        offset = (i - 1) * width  # -0.25, 0.0, +0.25 relative to each tick
        bars = ax.bar(x + offset, vals, width, label=lib, color=color, alpha=0.85)
        ax.bar_label(bars, fmt="%.3f", padding=3, fontsize=7)

    ax.set_xticks(x)
    ax.set_xticklabels(operations, rotation=30, ha="right", fontsize=9)
    ax.set_ylabel(title, fontsize=10)
    ax.legend(title="Library", fontsize=9)
    ax.grid(axis="y", linestyle="--", alpha=0.4)
    if metric == "throughput_rows_per_sec":
        ax.yaxis.set_major_formatter(
            mticker.FuncFormatter(lambda v, _: f"{v:,.0f}")
        )

    plt.tight_layout()
    plt.show()
```

Sample output:



This is the python code that generates a barchart containing all operations and libraries used. Each chart is grouped by the metrics used. For example, the figure shows the output for the execution time (s) metric.

6.0 Performance Evaluation

This section presents the evaluation of the performance in executing a list of operations for the three libraries implemented which were Pandas, Polars and PyArrow. The metrics used to compare their performance are execution time(s), memory used (MB), throughput (records/s) and CPU utilization (%).

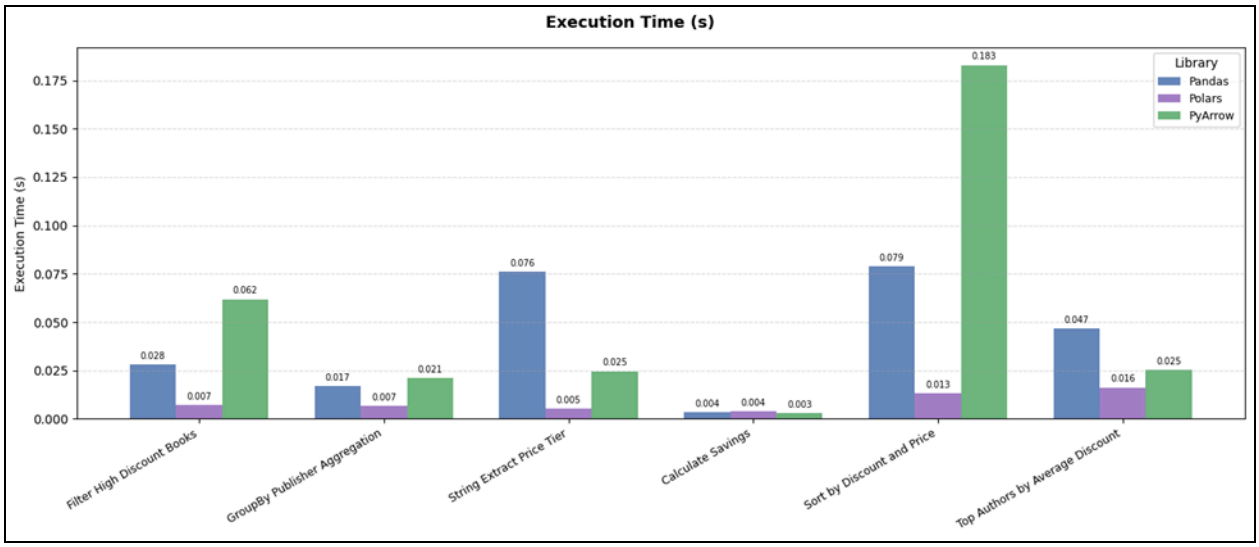
Operations used for benchmarking:

- Filter by price and discount (Operation 1)
- Group by publisher with aggregation (Operation 2)
- String extraction from tags column (Operation 3)
- Derive savings column (Operation 4)
- Sort by discount percentage and current price (Operation 5)
- Top authors by average discounts (Operation 6)

6.1 Query Time (s)

Query time measures how fast a program can execute an operation and return the results. A lower execution time is better as it reduces resource consumption and prevents system bottlenecks. The following table and figure shows the result of all operations for each library used.

Operation	**Pandas (s)**	**Polars (s)**	**PyArrow (s)**
Operation 1	0.028	0.007	0.062
Operation 2	0.017	0.007	0.021
Operation 3	0.076	0.005	0.025
Operation 4	0.004	0.004	0.003
Operation 5	0.079	0.013	0.183
Operation 6	0.047	0.016	0.025

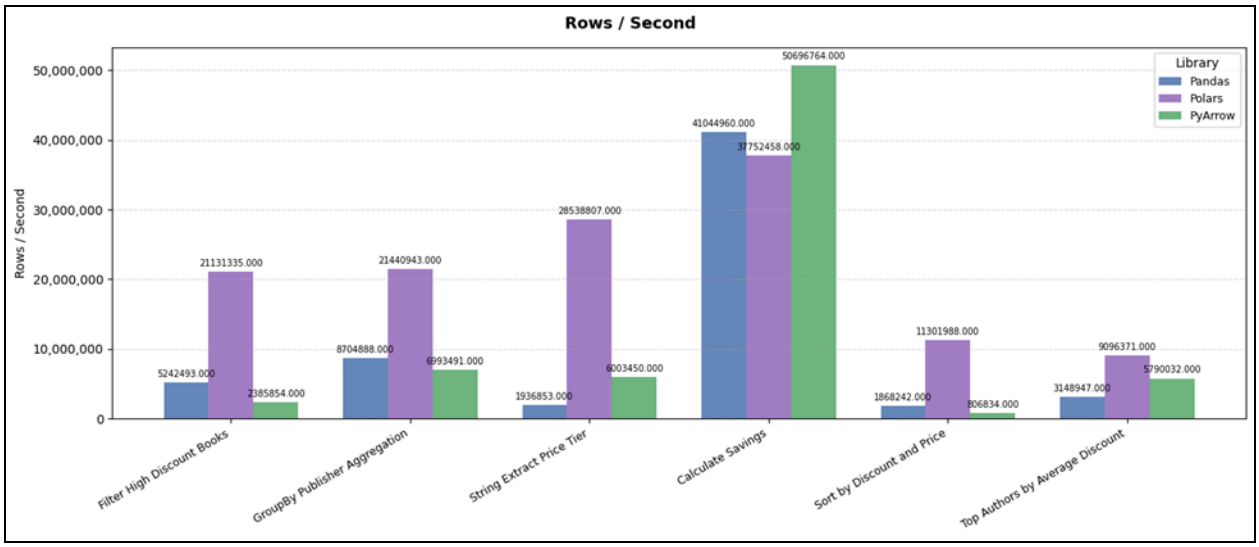


Based on the result above, Polars shows the best result compared to Pandas and PyArrows. Polars consistently completes the fastest across all operations with the exception of operation 4 having the same result as Pandas and PyArrow having lower result. This is due to Polars being built in Rust, known to be one of the fastest programming languages, that also utilizes all available CPU cores which enables multithreading and processes data in columns rather than rows. Pandas and PyArrow on the other hand show varying results. PyArrows shows the highest execution time with operation 5 which tests multi-key sorting almost 2 times more than Pandas. For aggregation operations, both Pandas and PyArrow are comparable but Pandas struggle more with operations that involve string operations.

6.2 Throughput (row/s)

Throughput measures the actual rate at which data is successfully transferred or processed over a network or system within a specific time frame which is linked to execution time.

Operation	**Pandas (rows/s)**	**Polars (rows/s)**	**PyArrow (rows/s)**
Operation 1	5,242,493	21,131,335	2,385,854
Operation 2	8,704,888	21,440,943	6,993,491
Operation 3	1,936,853	28,538,807	6,003,450
Operation 4	41,044,960	37,752,458	50,696,764
Operation 5	1,868,242	11,301,988	806,834
Operation 6	3,148,947	9,096,371	5,790,032

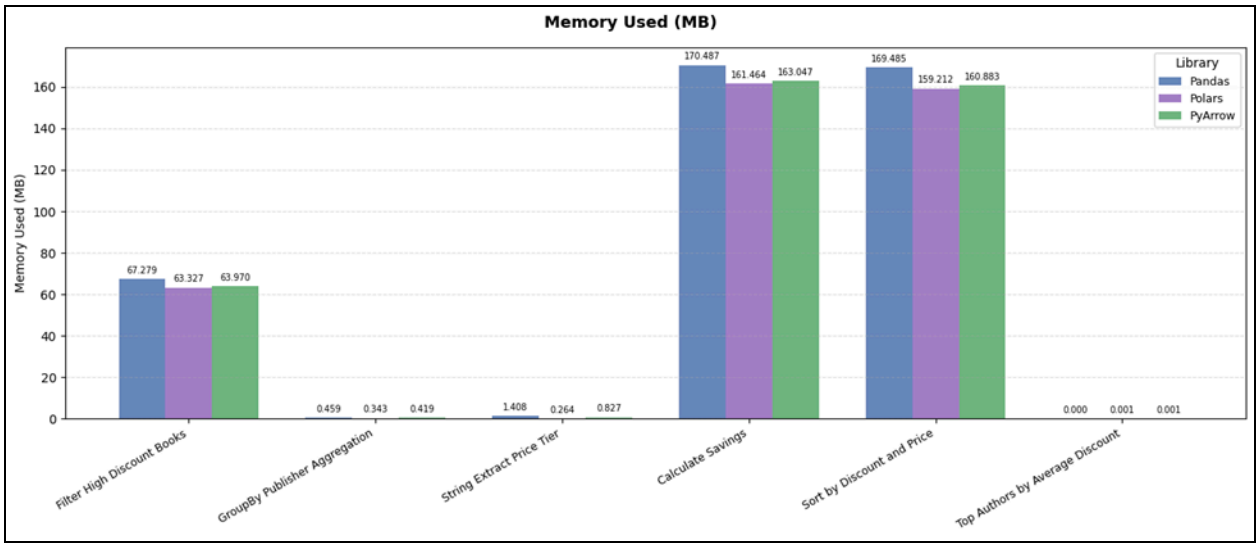


Overall, Polars shows consistently high throughput for almost all operations which is reflected by the result of the execution time. Pandas and PyArrow shows competitive result where some operations show Pandas having better throughput than PyArrow and some where PyArrow is higher.

6.3 Memory Usage (MB)

Memory usage refers to the in-memory footprint of each operation's result object, measured with each library's native sizing API.

Operation	Pandas (MB)	Polars (MB)	PyArrow (MB)
Operation 1	67.279	63.327	63.970
Operation 2	0.459	0.343	0.419
Operation 3	1.408	0.264	0.827
Operation 4	170.487	161.464	163.047
Operation 5	169.485	159.212	160.883
Operation 6	0.000	0.001	0.001

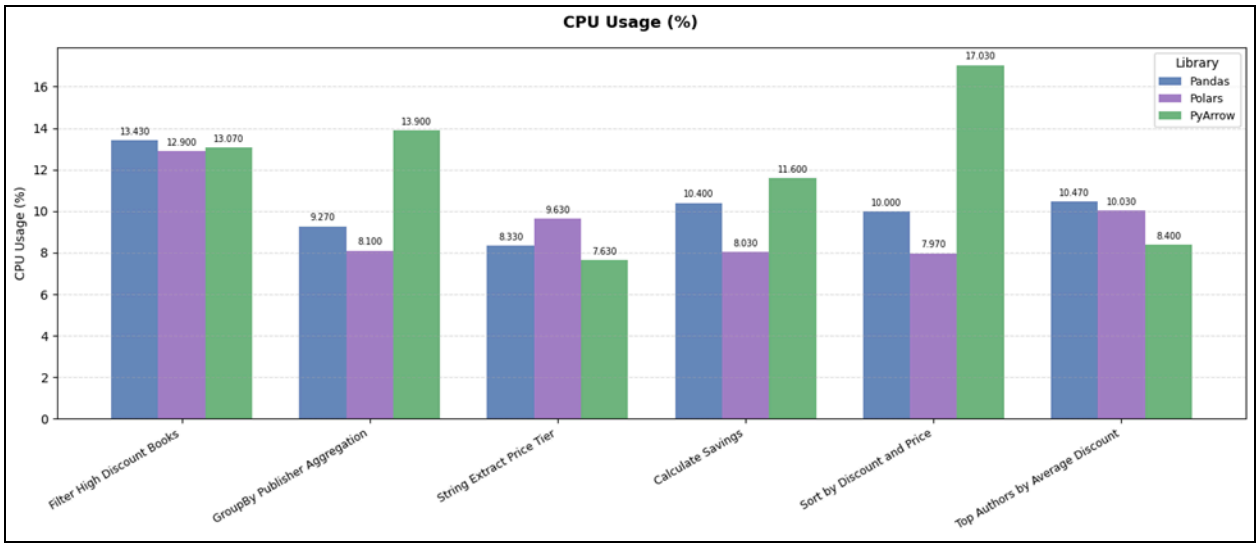


In terms of memory usage, the result does not show a significant difference in terms of memory usage for all three libraries. However, both Polars and PyArrow which use in-memory columnar storage show lower memory footprint compared to Pandas which suggests that Polars and PyArrow is more memory efficient than Pandas.

6.4 CPU Utilisation (%)

CPU usage is defined as the percentage of time processors spend in executing tasks or processing instructions. A normal CPU usage depends on the operation or task that the processor is trying to achieve. Operations used to test these libraries are considered low demand where the CPU usage should be below 20%. Thus, the usage should be around the range where the processor is not underutilized, slowing down execution time or overutilizing it which overloads the processor.

Operation	**Pandas (%)**	**Polars (%)**	**PyArrow (%)**
Operation 1	13.43	12.90	13.07
Operation 2	9.27	8.10	13.90
Operation 3	8.33	9.63	7.63
Operation 4	10.40	8.03	11.60
Operation 5	10.00	7.97	17.03
Operation 6	10.47	10.03	8.40



Overall, CPU usage across all three libraries is considered low and similar (~7% - 18%). Polars shows the best performance and efficiency because even with low CPU usage, it still delivers fast execution based on what was discussed previously. However, PyArrows CPU usage for operation 2 and 5 is shown to be high and is not reflected in a lower execution time which suggests that there is inefficiency in executing the operation. Operations 2, 3 and 6 that use almost zero memory show slightly higher CPU variance because they are more compute-intensive instead of data movement where the CPU must actually hash, compare, and aggregate rather than simply copy bytes.

7.0 Challenges and Limitations

- Mislabelled fields. The Shopify schema used “Product Type” for the author and “Vendor” for the publisher, which had to be identified and renamed.
- Easy to get blocked during scraping. The chosen website to be scraped need to be not having anti-bot protection smoother scraping. The CAPTCHA also becomes troublesome because the user needs to manually solve the CAPTCHA to continue scraping instead of making it fully automated
- Faster and low latency connectivity to the internet for faster scraping. Scraping needs to have low latency and fast internet to handle many HTTP requests to the website for smoother scraping.
- Data cleaning challenges. Data cleaning requires careful inspection of the data in order to determine which fields are dirty. Each column requires specific implementation in order to standardise the format.
- Fair and accurate benchmarking. Producing accurate readings from benchmarking requires flushing memory and taking clean CPU readings between runs and averaging multiple runs, since single measurements are noisy on short operations that often cause false readings.

8.0 Conclusion and Future Work

8.1 Summary of Findings

This project successfully collected and processed large-scale product data from BookXcess. The crawler used Beautiful Soup to parse the XML sitemap and retrieve product URLs. Each product's structured data was then collected through its public *.json* endpoint.

The raw dataset contained 154,714 rows, and after cleaning and removing duplicates, the final cleaned dataset contained 147,558 valid records. This meets the project requirement of collecting more than 100,000 structured records.

The data processing stage cleaned HTML descriptions, standardized column names, handled missing values, removed duplicate products, converted price fields, calculated discounts, and prepared the dataset for benchmarking using Pandas, PyArrow and Polars then evaluated in bar charts using matplotlib. It was found that Polars performed the best on average across all metrics for each operation.

8.2 What Could Be Improved

- **Save collected data into a database**
  Store the data in PostgreSQL, MySQL or MongoDB for better querying, indexing, and long-term storage.
- **Use Parquet instead of CSV**
  Parquet can reduce file size and improve reading speed for large-scale data processing.
- **Improve incremental crawling**
  Instead of scraping all products again, the crawler can collect only new or updated products.
- **Benchmark on larger datasets**
  Testing with 1 million or more records would give a clearer comparison between Pandas, PyArrow, and Polars.
- **Build a dashboard**
  The cleaned dataset can be visualised using Power BI, Tableau or Streamlit to analyse prices, discounts, publishers, and categories.

9.0 References

1. BookXcess Online Store. https://www.bookxcess.com
2. Polars — Lightning-fast DataFrame library. https://pola.rs
3. Apache Arrow / PyArrow Documentation. https://arrow.apache.org
4. pandas Documentation. https://pandas.pydata.org
5. curl_cffi — Python binding for curl-impersonate. https://github.com/yifeikong/curl_cffi
6. Beautiful Soup Documentation. https://www.crummy.com/software/BeautifulSoup/
7. psutil Documentation. https://psutil.readthedocs.io